

Stance measurement for one topic

```
suppressPackageStartupMessages({
  library(here); library(splines); library(viridis)
  library(tidyverse)      # LAST so its verbs win
})
# Namespaced only: ellmer (LLM calls), glue.

st <- list(
  topic = 1L, # THE topic this render processes
  open_router = TRUE, # FALSE on the work server
  dry_run = FALSE, # TRUE: build prompts, print arithmetic, stop
  pilot_n = NULL, # e.g. 20L to test on a subset first

  # One row per pipeline step. base_url_env is the NAME of the .Renvron
  # variable holding that step's endpoint, so different steps can point at
  # different gateways and no URL or key appears in this file. Ignored when
  # st$open_router is TRUE, which routes through llm_open_router_openrouter.R
  # instead.
  steps = tibble::tribble(
    ~step, ~base_url_env, ~api_key_env, ~model,
    "proposition", "COMPASS_LARGE_URL", "COMPASS_LARGE_KEY", "MODEL-FOR-PROPOSITIONS",
    "stance_1", "COMPASS_STANCE_URL", "COMPASS_STANCE_KEY", "MODEL-FOR-STANCE-1",
    "stance_2", "COMPASS_STANCE_URL", "COMPASS_STANCE_KEY", "MODEL-FOR-STANCE-2"),

  window_n = 300L, # calls allowed per quota window PER model
  window_wait = 4L * 60L * 60L, # seconds to sleep at the window boundary
  era_width = 5L, # year grouping for the era table
  df_spline = 3L, # spline df for the trend curve
  n_gold = 100L, # paragraphs sent for human coding
  rec_sizes = c(50, 100, 200, 400, 800), # recovery-curve subsample sizes
  rec_reps = 5L
)

theme_set(theme_minimal(base_size = 12))
# Tables must not silently lose rows in the rendered PDF.
options(knitr.table.max.print = Inf, knitr.kable.max_rows = Inf,
  max.print = 1e6)
set.seed(2026)

p_out <- function(f) here::here("outputs", f)
p_tag <- function(stem, ext = ".csv") # topic-stamped filenames
```

```

p_out(str_c(stem, "_topic", sprintf("%02d", st$topic), ext))
dir.create(here::here("outputs"), showWarnings = FALSE, recursive = TRUE)
source(here::here("code", "llm_source.R"))
if (isTRUE(st$open_router))
  source(here::here("code", "llm_openrouter.R"))
if (isTRUE(st$open_router))
  st$steps$model <- open_router_models[st$steps$step]

```

Frame: every paragraph in this topic

Script 1 assigned every paragraph its modal STM topic. This script measures stance on **all** paragraphs of one topic, so there is no sampling and no sampling error: the proportions below are exact for the corpus. The budget is handled by time rather than by design, classifying 300 paragraphs per quota window and sleeping until the reset, which makes the method portable to any corpus size without retuning a sample.

```

paragraph_theta <- readr::read_csv(p_out("paragraph_theta.csv"),
                                   show_col_types = FALSE)
topic_summary   <- readr::read_csv(p_out("topic_summary.csv"),
                                   show_col_types = FALSE)
stopifnot("Script 1 must supply modal topic and text" =
  all(c("para_uid", "atom_id", "Year", "text", "topic",
        "theta_max") %in% names(paragraph_theta)))

frame <- paragraph_theta |>
  dplyr::filter(topic == st$topic) |>
  dplyr::select(para_uid, atom_id, Year, n_tokens, text, theta_max)
if (nrow(frame) == 0) stop("No paragraphs for topic ", st$topic, ".")

n_windows <- ceiling(nrow(frame) / st>window_n)
glue::glue("Topic {st$topic}: {nrow(frame)} paragraphs from \\
  {dplyr::n_distinct(frame$atom_id)} comments across \\
  {dplyr::n_distinct(frame$Year)} years. \\
  {nrow(frame)} calls per model = {n_windows} quota window(s), \\
  about {round(n_windows * st>window_wait / 3600, 1)} hours \\
  unattended; the two raters run their windows independently.")

```

Topic 1: 181 paragraphs from 27 comments across 27 years. 181 calls per model = 1 quota window(s), about 4 hours unattended; the two raters run their windows independently.

Codebook: label, description, proposition

The codebook CSV written by Script 1 is the source of truth. Labels and descriptions come from there and are never redrafted here. The only field this script owns is the **proposition**, drafted once if the cell is empty and then left alone. All three fields enter the stance prompt, so **edit be-**

fore classifying: changing them afterward means the stored labels were produced by a different instrument. The script compares file times and warns if that has happened.

The proposition is the measurement target and must be a **contestable directional claim** (“funding for X should increase”), not a topic restatement (“this is about funding”), because a non-contestable proposition drives every paragraph to neutral and quietly destroys the measurement.

```
cb_path <- p_out("topic_codebook.csv")
if (!file.exists(cb_path))
  stop("No topic_codebook.csv in outputs/. Render Script 1 first; it drafts ",
       "the labels and descriptions this script needs.")
codebook <- readr::read_csv(cb_path, show_col_types = FALSE)
if (!st$topic %in% codebook$topic)
  stop("Topic ", st$topic, " is not in the codebook.")

# Labels and descriptions come from Script 1 and are never redrafted here.
# Only the proposition is this script's business, and only if it is empty.
if (is.na(codebook$proposition[codebook$topic == st$topic]) ||
    !nzchar(str_squish(codebook$proposition[codebook$topic == st$topic] %||%
""))) {
  row <- codebook |> dplyr::filter(topic == st$topic)
  sys <- build_prompt(
    role = str_c("You are a survey methodologist drafting stance items for a ",
                 "public research program."),
    task = str_c("From this topic's label and description, draft ONE ",
                 "proposition that a commenter could plausibly support or ",
                 "oppose. Before answering, test it: if a paragraph could not ",
                 "disagree with it, rewrite it."),
    rules = c("A proposition states a direction, not a subject.",
              "Use only the label and description shown; invent no programs.",
              "Keep it under 20 words and free of loaded language.",
              "Do not restate the topic label as a proposition."),
    output = "One proposition.")
  schema <- ellmer::type_object(
    proposition = ellmer::type_string("A contestable directional claim.))
  draft <- call_one(make_chat("proposition", sys, st$steps, st$open_router),
                    str_c("Label: ", row$label,
                          "\nDescription: ", row$description), schema)
  if (is.null(draft)) stop("Proposition drafting call failed for topic ",
                          st$topic, ".")
  codebook$proposition[codebook$topic == st$topic] <- draft$proposition
  readr::write_csv(codebook, cb_path)
  glue::glue("DRAFT proposition written for topic {st$topic}. Edit \\
             {basename(cb_path)} before classifying: it is the measurement \\
             target and it enters every stance prompt.")
}

cb <- codebook |> dplyr::filter(topic == st$topic)
```

```
stopifnot("codebook row must be complete" =
  all(nzchar(c(cb$label, cb$description, cb$proposition))))
cb |>
  dplyr::mutate(ttl = str_c("Topic ", topic, ": ", label)) |>
  tbl_records("ttl", c("description", "proposition"),
    caption = "Codebook row driving this render. All three fields
enter the stance prompt; the proposition is the sole measurement target.")
```

Codebook row driving this render. All three fields enter the stance prompt; the proposition is the sole measurement target.

Topic 1: World Peace - description: These paragraphs are about promoting peace and freedom in the world among nations. - proposition: Nations should prioritize cooperation over competition to achieve global stability.

The instrument

Every prompt has four parts. **ROLE** frames the expertise. **TASK** states the job, gives the topic context from the codebook, and asks for a self-check. **RULES** is the anti-hallucination slot and carries the line that keeps the label from becoming a second target. **OUTPUT** states the shape and is paired with a structured schema, so an unparseable answer is impossible rather than merely discouraged. Year is deliberately withheld: telling a rater the period invites it to infer position from context rather than read it from text.

```
stance_labels <- c("favor", "neutral", "oppose", "irrelevant")
stance_schema <- ellmer::type_object(
  label = ellmer::type_enum(stance_labels, "The single best-fitting label.))

stance_sys <- build_prompt(
  role = str_c("You are a survey methodologist coding stance in public ",
    "comments for a research program."),
  task = str_c("These paragraphs were grouped under the topic \"", cb$label,
    "\": ", cb$description, " Decide the author's position toward ",
    "this proposition: \"", cb$proposition, "\". First check ",
    "whether the text addresses the proposition at all; if it ",
    "does not, the label is irrelevant. Then decide favor, ",
    "oppose, or neutral."),
  rules = c(str_c("Stance is measured ONLY toward the proposition. The topic ",
    "label and description are context for judging relevance, ",
    "not a second target."),
    "Stance is not tone: an angry passage can favor the proposition.",
    "Judge only the text shown; use no outside knowledge.",
    "If the text is ambiguous, choose neutral rather than guessing."),
  output = "One label: favor, neutral, oppose, or irrelevant.")

cat("`\\n", stance_sys, "\\n`\\n", sep = "")
```

ROLE

You are a survey methodologist coding stance in public comments for a research program.

TASK

These paragraphs were grouped under the topic "World Peace": These paragraphs are about promoting peace and freedom in the world among nations. Decide the author's position toward this proposition: "Nations should prioritize cooperation over competition to achieve global stability.". First check whether the text addresses the proposition at all; if it does not, the label is irrelevant. Then decide favor, oppose, or neutral.

RULES

- Stance is measured ONLY toward the proposition. The topic label and description are context for judging relevance, not a second target.
- Stance is not tone: an angry passage can favor the proposition.
- Judge only the text shown; use no outside knowledge.
- If the text is ambiguous, choose neutral rather than guessing.

OUTPUT

One label: favor, neutral, oppose, or irrelevant.

Classification

Sequential, one paragraph per call: items sharing a context bleed into each other, and stance is where that matters most. Two raters, each with its own quota, classified in blocks of 300 with a sleep at the window boundary. The label store is **incremental**, so an interrupted render, a crash, or a rerun re-sends only what is missing; a temperature-0 label is a fixed measurement of a fixed paragraph.

```
todo_all <- if (is.null(st$pilot_n)) frame else
  dplyr::slice_head(frame, n = st$pilot_n)

if (isTRUE(st$dry_run)) {
  glue::glue("DRY RUN: {nrow(todo_all)} paragraphs x \\
    {sum(str_detect(st$steps$step, '^stance'))} raters = \\
    {nrow(todo_all) * sum(str_detect(st$steps$step, '^stance'))} \\
    calls. No requests made. Set st$dry_run = FALSE to run.")
  knitr::knit_exit()
}

store_path <- p_tag("stance_labels")
store <- if (file.exists(store_path))
  readr::read_csv(store_path, show_col_types = FALSE) else
  tibble(para_uid = character(), rater = character(), label = character())

if (file.exists(store_path) &&
```

```

file.mtime(cb_path) > file.mtime(store_path))
warning("The codebook was edited after these labels were produced. The ",
        "stored labels came from a different prompt; delete ",
        basename(store_path), " and reclassify.")

raters <- st$steps |> dplyr::filter(str_detect(step, "^stance"))
new <- purrr::map(raters$step, function(r) {
  todo <- todo_all |>
    dplyr::anti_join(dplyr::filter(store, rater == r), by = "para_uid")
  if (nrow(todo) == 0) return(NULL)
  chat <- make_chat(r, stance_sys, st$steps, st$open_router)
  classify_census(chat, str_c("Text: ", todo$text), todo$para_uid,
                  stance_schema, st>window_n, st>window_wait) |>
    dplyr::transmute(para_uid = id, rater = r, label)
}) |> purrr::list_rbind()

store <- dplyr::bind_rows(store, new) |>
  dplyr::distinct(para_uid, rater, .keep_all = TRUE)
readr::write_csv(store, store_path)
glue::glue("{dplyr::n_distinct(store$para_uid)} paragraphs labeled by \\
            {dplyr::n_distinct(store$rater)} raters; {nrow(new)} new this \\
            render; {sum(is.na(store$label))} unresolved (re-render to retry).")

```

181 paragraphs labeled by 2 raters; 0 new this render; 0 unresolved (re-render to retry).

Reliability

Agreement between raters is reliability, not accuracy: both can be wrong together. Kappa is reported with raw agreement and the confusion table, because kappa alone has no validated interpretive thresholds.

```

wide <- store |> dplyr::filter(!is.na(label)) |>
  tidyr::pivot_wider(names_from = rater, values_from = label)
ag <- agreement(wide[[2]], wide[[3]], dnn = names(wide)[2:3])
glue::glue("Agreement over {ag$n} paragraphs: kappa = {round(ag$kappa, 3)}, \\
            raw = {round(ag$raw, 3)}")

```

Agreement over 181 paragraphs: kappa = 0.467, raw = 0.663

```
ag$confusion
```

	stance_2
stance_1	favor irrelevant neutral oppose

favor	83	0	4	1
irrelevant	21	31	30	3
neutral	1	0	2	1
oppose	0	0	0	4

Human coding

The only thing that turns “the raters agreed” into “the raters were right”. The coding file shows the coder the same label, description, and proposition the rater saw, and no model output, so the comparison is not anchored.

```
gold_out <- p_tag("gold_to_code")
frame |>
  dplyr::slice_sample(n = min(st$n_gold, nrow(frame))) |>
  dplyr::transmute(para_uid, topic = st$topic, topic_label = cb$label,
                    topic_description = cb$description,
                    proposition = cb$proposition, text,
                    human_label = NA_character_, coder_notes = NA_character_) |>
  readr::write_csv(gold_out)

gold_in <- p_tag("gold_coded")
if (file.exists(gold_in)) {
  gold <- readr::read_csv(gold_in, show_col_types = FALSE)
  purrr::walk(unique(store$rater), function(r) {
    j <- store |> dplyr::filter(rater == r) |>
      dplyr::inner_join(dplyr::select(gold, para_uid, human_label),
                        by = "para_uid")
    m <- class_metrics(j$label, j$human_label)
    k <- agreement(j$label, j$human_label, dnn = c(r, "human"))
    print(glue::glue("[{r}] macro-F1 {round(m$macro_f1, 3)}, accuracy \\
                      {round(m$accuracy, 3)}, kappa vs human \\
                      {round(k$kappa, 3)}"))
    tbl_paged(m$per_class, caption = str_c("Per-class validity, ", r))
  })
} else {
  glue::glue("No coded file yet. Code {basename(gold_out)}, save it as \\
             {basename(gold_in)} with human_label filled, and re-render. \\
             Until then the stance labels are unvalidated.")
}
```

No coded file yet. Code gold_to_code_topic01.csv, save it as gold_coded_topic01.csv with human_label filled, and re-render. Until then the stance labels are unvalidated.

Results

```

est <- frame |>
  dplyr::inner_join(store |> dplyr::filter(rater == raters$step[1],
                                           !is.na(label)),
                    by = "para_uid") |>
  dplyr::mutate(oppose = as.integer(label == "oppose"),
               era = str_c(floor((Year - min(Year)) / st$era_width) *
                           st$era_width + min(Year), "s"))

est |>
  dplyr::count(era, label) |>
  dplyr::group_by(era) |>
  dplyr::mutate(p = round(n / sum(n), 3)) |>
  dplyr::ungroup() |>
  tidyr::pivot_wider(id_cols = era, names_from = label, values_from = p,
                     values_fill = 0) |>
  tbl_paged(caption = "Stance proportions by era. Every paragraph in the topic
was classified, so these are exact for the corpus and carry no sampling error;
the live uncertainty is rater misclassification, quantified by the human-coded
metrics above.")

```

Stance proportions by era. Every paragraph in the topic was classified, so these are exact for the corpus and carry no sampling error; the live uncertainty is rater misclassification, quantified by the human-coded metrics above.

era	favor	irrelevant	neutral	oppose
1921s	0.615	0.231	0.077	0.077
1926s	0.667	0.333	0.000	0.000
1931s	0.000	1.000	0.000	0.000
1936s	0.167	0.833	0.000	0.000
1941s	0.333	0.667	0.000	0.000
1946s	0.412	0.588	0.000	0.000
1951s	0.500	0.417	0.000	0.083
1956s	0.667	0.333	0.000	0.000
1961s	0.733	0.267	0.000	0.000
1966s	0.636	0.364	0.000	0.000
1971s	0.571	0.286	0.143	0.000
1976s	0.250	0.625	0.125	0.000
1981s	0.429	0.500	0.000	0.071
1986s	0.000	1.000	0.000	0.000

era	favor	irrelevant	neutral	oppose
1991s	0.000	1.000	0.000	0.000
1996s	0.000	1.000	0.000	0.000
2001s	0.333	0.667	0.000	0.000
2006s	1.000	0.000	0.000	0.000
2011s	1.000	0.000	0.000	0.000
2016s	0.000	0.000	1.000	0.000

Stance proportions by era. Every paragraph in the topic was classified, so these are exact for the corpus and carry no sampling error; the live uncertainty is rater misclassification, quantified by the human-coded metrics above. (continued, 2 of 2)

era	favor	irrelevant	neutral	oppose
2021s	0.25	0.75	0	0

The trend curve is a different kind of statement. It treats the corpus as one realization of a comment-generating process, so its interval is model-based, not design-based, with **cluster-robust standard errors on the comment** because paragraphs from one author are correlated. The exact era proportions above need no interval; the curve does, and the two should not be mixed in a single sentence.

```
rel <- est |> dplyr::filter(label != "irrelevant")
stances <- c("favor", "neutral", "oppose")

# What each class actually has to work with. Read this before the plot: a class
# with few positives or few distinct years cannot support a spline, and the
# panel will show observed points only.
counts <- rel |>
  dplyr::filter(label %in% stances) |>
  dplyr::group_by(stance = label) |>
  dplyr::summarise(n = dplyr::n(), years = dplyr::n_distinct(Year),
                  .groups = "drop")
print(counts)
```

```
# A tibble: 3 × 3
  stance      n years
<chr>   <int> <int>
1 favor     88    20
2 neutral    4     4
3 oppose     4     3
```

```

# Observed yearly share per stance, computed for all three regardless of fit.
obs <- rel |>
  dplyr::count(Year, label) |>
  dplyr::group_by(Year) |>
  dplyr::mutate(share = n / sum(n)) |>
  dplyr::ungroup() |>
  dplyr::filter(label %in% stances) |>
  dplyr::mutate(stance = factor(label, levels = stances))

# Fit each class, reporting failures rather than dropping the class.
fit_one <- function(s) {
  d <- rel |> dplyr::mutate(y = as.integer(label == s))
  out <- tryCatch(trend_curve(d, "y", "atom_id", st$df_spline),
    error = function(e) {
      message("[", s, "] no spline fit: ", conditionMessage(e))
      NULL
    })
  if (is.null(out)) NULL else dplyr::mutate(out, stance = s)
}
tr <- purrr::map(stances, fit_one) |> purrr::compact() |> purrr::list_rbind()
if (nrow(tr)) {
  tr <- dplyr::mutate(tr, stance = factor(stance, levels = stances))
  readr::write_csv(tr, p_tag("stance_trend"))
}

p <- ggplot(mapping = aes(Year)) +
  geom_point(data = obs, aes(y = share, color = stance), alpha = .45,
    size = 1) +
  facet_wrap(~ stance, ncol = 3, drop = FALSE) +
  scale_color_viridis_d(end = .85, drop = FALSE, guide = "none") +
  scale_fill_viridis_d(end = .85, drop = FALSE, guide = "none") +
  scale_y_continuous(limits = c(0, 1)) +
  labs(x = NULL, y = "share of relevant paragraphs", title = cb$label,
    subtitle = glue::glue("{nrow(rel)} of {nrow(est)} paragraphs address the
proposition"))
if (nrow(tr))
  p <- p +
    geom_ribbon(data = tr, aes(ymin = lo, ymax = hi, fill = stance),
      alpha = .2) +
    geom_line(data = tr, aes(y = fit, color = stance), linewidth = .8)
print(p)

```

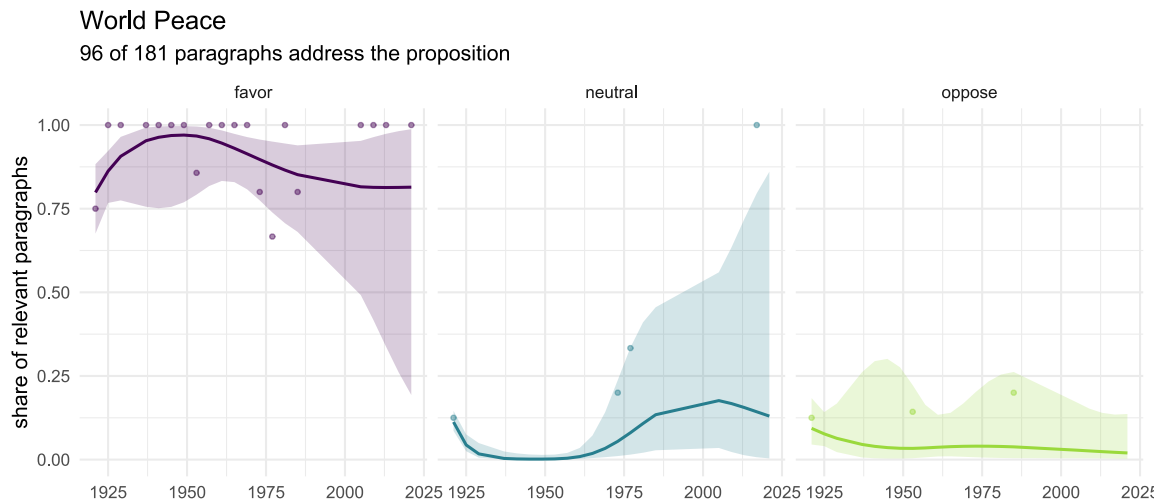


Figure 1: Share of relevant paragraphs in each stance, by year. Lines are cluster-robust spline fits with 95 percent intervals where the class supports one; points are the observed yearly shares, shown in every panel so a sparse class is still visible.

How many paragraphs would have sufficed?

Because the topic was censused, this costs nothing: subsample the labeled paragraphs at increasing sizes, refit the trend, and measure how far each fitted curve sits from the full-census curve. Comments are subsampled rather than paragraphs, so each replicate has the clustered shape a real collection would have. Where the curve flattens is the point past which more classification buys little, and that number, not a formula, is what should size the next corpus.

```
rec <- recovery_curve(est, "oppose", st$rec_sizes[st$rec_sizes < nrow(est)],
                      st$rec_reps, st$df_spline)
readr::write_csv(rec, p_tag("recovery_curve"))
ggplot(rec, aes(n_actual, mad)) +
  geom_point(alpha = 0.4, color = viridis(1, begin = 0.35)) +
  geom_smooth(se = FALSE, color = viridis(1, begin = 0.6), linewidth = 0.8) +
  labs(x = "paragraphs classified", y = "mean absolute deviation from census")
```

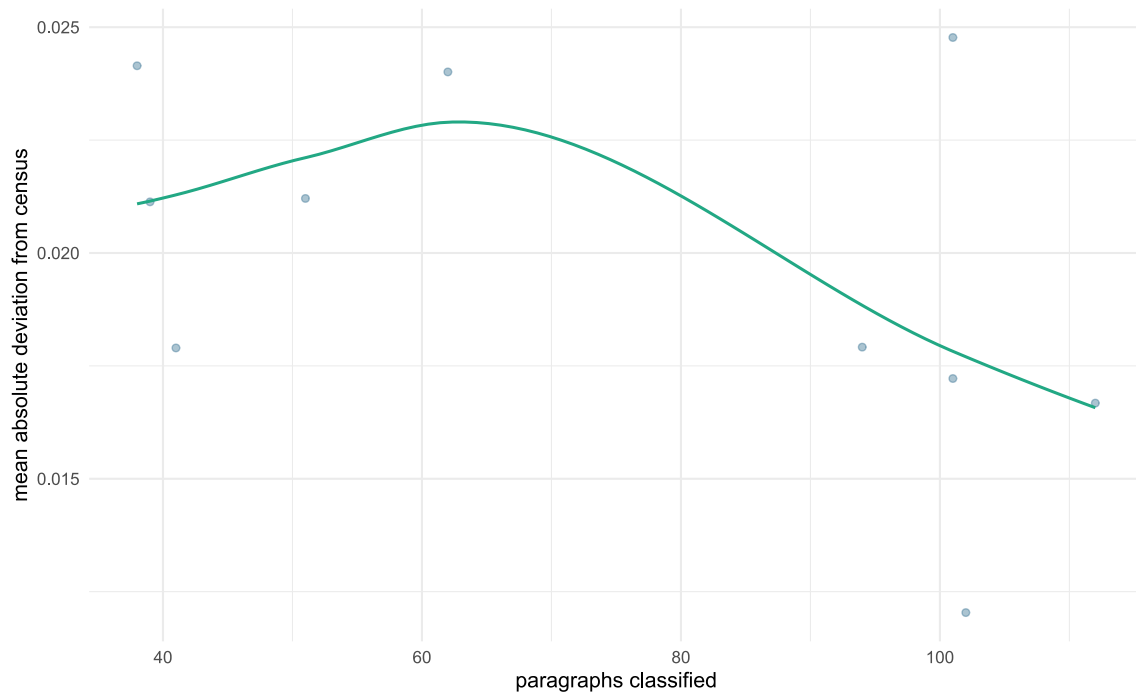


Figure 2: Mean absolute deviation from the full-census trend curve, by subsample size. Points are replicates; the line is the mean. The flattening point is the practical sample size for this topic.

Documentation

```
emit_data_dict(
  path = p_tag("data-dict", ".yaml"),
  name = str_c("stance_topic_", st$topic),
  description = str_c("Census stance measurement for STM topic ", st$topic,
    " (", cb$label, ")."),
  tables = list(
    list(name = "topic_codebook", path = cb_path, data = codebook,
      description = "Label, description, and proposition per topic. Analyst-
        edited; all three enter the stance prompt.",
      vars = list(topic = "STM topic number", label = "Short human-readable
        name",
          description = "One factual sentence about the topic",
          proposition = "The contestable claim stance is measured
            toward")),
    list(name = "stance_labels", path = store_path, data = store,
      description = "One row per paragraph per rater; the incremental label
        store.",
      vars = list(para_uid = "Paragraph id", rater = "Pipeline step that
        produced the label",
          label = "favor, neutral, oppose, or irrelevant")),
    list(name = "stance_trend", path = p_tag("stance_trend"), data = tr,
```

```

description = "Fitted P(oppose) by year with cluster-robust intervals.",
vars = list(Year = "Calendar year", fit = "Fitted probability",
            lo = "Lower 95% bound", hi = "Upper 95% bound",
            se_ratio = "Robust/naive SE ratio; the clustering cost",
            stance = "Which stance this curve estimates")),
list(name = "recovery_curve", path = p_tag("recovery_curve"), data = rec,
description = "Deviation from the census trend by subsample size.",
vars = list(n_target = "Requested subsample size", rep = "Replicate
index",
            n_actual = "Paragraphs actually drawn", mad = "Mean absolute
deviation"))),
glossary = list(
  stance = "Position toward a stated proposition (favor, neutral, oppose) or
irrelevant; distinct from sentiment.",
  proposition = "The contestable claim stance is measured toward; analyst-
authored.",
  census = "Every paragraph in the topic was classified, so proportions are
exact and carry no sampling error.",
  `cluster-robust` = "Standard errors allowing correlation among paragraphs
from the same comment.",
  `recovery curve` = "Post-hoc subsampling of the census showing how quickly
the trend estimate stabilizes."))

glue::glue("Wrote: {basename(store_path)}, {basename(p_tag('stance_trend'))}, \
\
            {basename(p_tag('recovery_curve'))}, {basename(gold_out)}, \
{basename(p_tag('data-dict', '.yaml'))}. Raters: \
{str_c(raters$model, collapse = ' | ')}; temperature 0; \
run {format(Sys.Date())}.")

```

Wrote: stance_labels_topic01.csv, stance_trend_topic01.csv,
recovery_curve_topic01.csv, gold_to_code_topic01.csv, data-dict_topic01.yaml.
Raters: openrouter/google/gemma-4-31b-it | openrouter/meta-llama/llama-3.1-70b-
instruct; temperature 0; run 2026-08-07.

Limitations

1. **Exact does not mean correct.** A census removes sampling error, not measurement error. Rater misclassification is the live uncertainty and only the human-coded metrics speak to it.
2. **Modal topic assignment carries error.** Topics are model-derived domains; a paragraph near the boundary between two topics is assigned as confidently as one at the center.
3. **Two framings, one document.** Era proportions are exact for this corpus; the trend interval is a superpopulation statement. Do not combine them in one claim.
4. **Agreement is not accuracy.** Two raters agreeing shows reliability, and both can be wrong in the same direction.

5. **The instrument includes the codebook.** Label, description, and proposition are in the prompt, so editing them after a run invalidates the stored labels; the script warns, but the discipline is yours.